

Technical report - Apertus FlowGuard

- Track: **Track 2B - Own Project**
- Event: Hack Apertus Online 2026
- Entrant: individual participant based in France
- Demo: not yet recorded; a video of at most two minutes is required for submission
- Status: local prototype with Docker structured demo verified; live Apertus inference still requires verification

1. Summary

Laboratory capillary users often ask for a pressure estimate in mixed natural-language units. FlowGuard uses Apertus v1.5 only to extract explicitly stated quantities and assumptions; a separate, deterministic Python module converts units, checks the physical domain and computes pressure loss. It refuses to give a numerical answer for gases, non-Newtonian fluids, missing properties, turbulence, unknown geometry, fittings or short entrance lengths. In the built-in synthetic example, its result is **3,395.3 Pa**, with **Re = 2.118**. This is a transparent screening estimate, not a certified design value.

2. Architecture

question → Apertus v1.5 JSON extraction → quote/value/unit checks → deterministic domain gate → Hagen-Poiseuille solver → structured JSON result. The direct structured-input path skips the model and exercises the same solver. Details are in ``docs/architecture.md``.

The container uses Python's standard library only. No uploaded dataset, model weights, tracking service or external package is required by the application. An Apertus v1.5 inference endpoint is required for natural-language mode. It receives the user's question; users should not submit personal, confidential or proprietary text to an endpoint without checking its operator and data policy. The app does not persist questions or API keys.

Target architecture (mandatory)

On-premise is the selected deployment target: run this small application container alongside an organization-managed Apertus v1.5 OpenAI-compatible inference service, with `LLM_BASE_URL` pointing to that service. The model weights and inference server are separate from this repository; their capacity and installation are not demonstrated here. **Air-gapped** operation would also be possible if the Python base image and Apertus inference service are preloaded inside the disconnected network, but has not been tested. A hosted endpoint outside Swiss jurisdiction does **not** by itself satisfy the sovereign Swiss-cloud option. Build-time dependencies: Docker and a locally available or pulled `python:3.12-alpine` image. Runtime dependencies: the Python image plus the optional reachable Apertus endpoint for question mode.

3. Use of Apertus

- Intended model: `swiss-ai/Apertus-v1.5-8B` or a compatible Apertus v1.5 endpoint.
- Role: extraction of *user-provided* numerical values, units and stated physical assumptions; never final calculation or physical-property lookup.
- Serving: OpenAI-compatible `/chat/completions`, selected through `LLM_NAME`, `LLM_BASE_URL`, `LLM_API_KEY`.
- Rebuilding: set those environment variables privately and `FLOWGUARD_QUESTION`, then run `make run`. Temperature is 0; response must be a single JSON object with the documented schema. The application checks a verbatim evidence quote for every quantity and rejects a value or unit inconsistent with its quote. The deterministic solver checks units and physical assumptions again.

No fine-tuning, adapter or quantization is included. The precise hosted provider, model identifier and inference response have **not yet been verified with credentials**; that validation is required before submission. When no endpoint is configured, `make run` demonstrates the structured path only and explicitly labels it as such.

4. Data

The built-in case is synthetic: density 998 kg/m³, dynamic viscosity 1 mPa·s, straight circular capillary length 5 cm, inner diameter 100 μm and volume flow 10 μL/min, all explicitly declared as a Newtonian incompressible liquid with no minor losses. These are illustrative inputs, not measured validation data. No personal data or third-party dataset is bundled. The separate `data/` directory holds the evaluation cases and their provenance, subject to the 100 MB limit.

5. Evaluation

The predeclared evaluation protocol and twelve frozen synthetic test questions are in `evaluation/` and `data/`. Scores are exact accept/refuse accuracy, unsafe acceptances among the eight out-of-scope prompts, relative errors in pressure and Reynolds number on the four valid prompts, latency and request count. The target is zero unsafe acceptances and at least three valid cases within 2% of the analytical references. The independent baseline asks the **same Apertus model to answer each question directly, without the deterministic calculator**, then scores it by the same rules. The structured demo is only a solver check, not that baseline. **No live hosted-Apertus accuracy or field-validation claim is made yet.**

For the synthetic structured example, the point pressure loss is **3,395.305 Pa** and **Re** is **2.11782**. With illustrative relative input uncertainty of 1% in density, 2% in viscosity, 1% in length, 2% in diameter and 2% in flow, the deterministic extreme-input pressure interval is **[2,982.4, 3,868.1] Pa**. This interval covers only the supplied input bounds; it excludes error in the flow model, entrance effects, calibration and measurement bias. If any uncertainty is omitted, the application reports `not_quantified`.

6. Limitations

The pressure equation applies to steady, fully developed laminar flow of a Newtonian incompressible liquid in a straight circular tube. $Re < 2300$ and $L \geq 0.05 Re d$ are screening checks, not proof of the assumptions. The entrance-length relation is approximate. The pressure estimate excludes entrance and exit losses, fittings, bends, compliance, nonuniform diameter, temperature effects and model discrepancy. Such conditions can change the true pressure loss even when Re is small. Results must not be used to certify a pump, pressure vessel, patient-contact device or other safety-critical system.

Apertus extraction can misread language; an exact quote and numeric/unit consistency check catches some but not all errors. Live inference and testing against independent measured capillary data remain outstanding. An on-premise Apertus service may require hardware well beyond the current development laptop.

7. Reproducibility

From repository root, `make test` runs 15 targeted numerical, domain, transport and extraction checks using Python 3. `make run` built and ran the synthetic demo in Docker on 1 October 2026; no secret was required. The default case has no stochastic component. The application is dependency-free beyond Python's standard library. For natural-language inference, record the exact Apertus model checkpoint, serving stack, endpoint location and prompt revision. No repository commit has been frozen yet; record it when preparing the submission. A clean-checkout Docker run is still required.

8. Next steps

Run the frozen French and English benchmark against the actual Apertus endpoint; inspect extraction errors; repeat the benchmark on an untouched second set; compare estimated pressure with independent measured capillary data; quantify model discrepancy and calibration; test an on-premise deployment and a clean-checkout `make run`.

License

Source code: Apache License 2.0 (`./LICENSE`). Report text: CC BY 4.0, consistent with the Track 2B report template. Project is new and contains no Jarvis source.

References

1. Olsen, L. O., and Ruegg, F. W., *An In-line Density and Viscosity Sensor*, NBSIR 74-620 (1974), section 2.1: straight circular tube, steady laminar regime and Hagen-Poiseuille relation. <https://nvlpubs.nist.gov/nistpubs/Legacy/IR/nbsir74-620.pdf>
2. Cimbala, J. M., Penn State ME 320 lecture notes, hydrodynamic entrance length for laminar pipe flow, $L_e/D \approx 0.05 Re$. https://www.me.psu.edu/cimbala/me320web_Spring_2015/Lectures/Tablet_PC_notes/ME320_Lecture_20.pdf